GavelLive — Build Roadmap

> This is **Document 2 of 2**. It tells you *how* to build GavelLive, one phase at a time. Read `GavelLive-01-Project-Brief.md` first to understand *what* you're building.

How to use this document

Build **strictly in order**, Phase 0 → Phase 10. Each phase ends with a **Definition of Done**. Do not move on until every box is ticked — that rule is what stops you getting lost in the middle.

The golden rule of the whole project:

> **Never add a technology before you've felt the problem it solves.** You add Redis *after* reads feel slow. You add the message broker *after* a notification blocks a bid. Tech added to fix a pain you've personally felt is tech you remember forever. Tech added because a doc said so is forgotten by Friday.

Every phase below uses the same five headings so you always know where you are:

- **Goal** — the one thing this phase achieves
- **Build** — what you actually make
- **Why / Where / How** — for each new technology: *why* it exists, *where* it plugs into GavelLive, *how* you wire it in
- **Definition of Done** — the checklist to pass before moving on
- **Pitfalls** — the beginner traps to avoid

A note on code snippets: they are **illustrative shapes**, not copy-paste-ready production code, and APIs change. For exact, current syntax (especially Aspire and MassTransit), pull the official docs at the moment you build that phase — links are in §"Resources" at the end.

Phase 0 — Setup & foundations

Goal: A clean, empty, version-controlled starting point and a clear plan for the first three features.

Build:

- Install the toolbelt: the .NET SDK, Node.js + the Angular CLI, Docker Desktop, Git, and an editor (VS Code or Rider/Visual Studio).
- Create a Git repository. First commit: this brief + roadmap, so your plan lives *in* the repo.
- Create the solution skeleton: one .NET Web API project, one Angular app, in a sensible folder layout (e.g. `/src/Api`, `/src/web`).

- Confirm both run: the API returns the default sample endpoint; the Angular app shows its default page.
- Write down your first three slices on paper: **CreateAuction**, **GetAuctions** (list), **PlaceBid**.

Why / Where / How: No new "stack" tech here — this phase is about removing every "does my machine even work" obstacle *before* you're also wrestling with a new concept. Confidence compounds.

Definition of Done:

- ☐ `dotnet --version`, `node --version`, `ng version`, `docker --version`, `git --version` all work
- ☐ The empty API runs and you can hit it in a browser/Swagger
- ☐ The empty Angular app runs in the browser
- ☐ Everything is committed to Git

Pitfalls:

- Spending a week "perfecting" tooling/themes. Get it running and move on.
- Skipping Git. You want to be able to undo confidently from Phase 1.

Phase 1 — The core canvas

Goal: A working end-to-end app where you can create an auction, see a list, and place a bid — the *dumb* way, with a page refresh to see changes. No real-time, no fancy infrastructure.

Build:

- A **PostgreSQL** database with `Auctions` and `Bids` tables (via EF Core, see below).
- Three API endpoints matching your three slices: create an auction, list auctions, place a bid.
- Bidding logic at its simplest: write the bid, set the auction's `CurrentPrice`, return the new price.
- An Angular page that lists auctions and a detail page with a "Place Bid" box. After you bid, you **refresh** to see the new price. Ugly on purpose.

Why / Where / How — PostgreSQL + EF Core

- **Why:** Your app needs a permanent, reliable source of truth for auctions and bids. Postgres is a free, production-grade relational database. **EF Core** (Entity Framework Core) is the .NET library that lets you work with database rows as C# objects instead of writing raw SQL.
- **Where:** It sits behind your API. Every command (place a bid) and query (list auctions) ultimately reads/writes Postgres in these early phases.
- **How:** Define C# classes for `Auction` and `Bid` (matching the brief's §4). Create an EF Core `DbContext` that exposes them. Use EF Core *migrations* to create the actual tables (`dotnet ef migrations add Initial`, then `dotnet ef database update`). For now run Postgres however is easiest — a local install or a single `docker run postgres`. (Phase 4 makes this clean.)

Definition of Done:

- ☐ You can create an auction via the API and see it stored in Postgres

- [ ] The Angular list page shows real auctions from the database
- [ ] You can place a bid and, after refreshing, see the updated price
- [ ] The first business rule works: a bid below the current price is rejected

Pitfalls:

- Building 15 features. Build exactly three. The canvas only needs to prove the pipe works end to end.
- Over-engineering the bid logic now — you *want* it naive so Phase 2 can reveal the concurrency bug.

Phase 2 — Vertical Slices + MediatR + CQRS (the spine)

Goal: Re-shape your code so each feature is a self-contained slice, and meet CQRS and the concurrency bug.

Build:

- Reorganize the three features into a `Features/` folder, one folder per feature (`CreateAuction`, `GetAuctions`, `PlaceBid`), each containing its request, validation, handler, and endpoint.
- Introduce **MediatR** so a thin endpoint just hands a request to its handler.
- Label each slice honestly: `CreateAuction` and `PlaceBid` are **commands** (writes); `GetAuctions` is a **query** (read). That labelling *is* CQRS.
- **Deliberately trigger the concurrency bug** (brief §6.1): place two bids at nearly the same time and watch the price logic misbehave. Then fix it with **optimistic concurrency** (add a version/row-version to the Auction; EF Core rejects a stale save; you retry).

Why / Where / How — Vertical Slice Architecture

- **Why:** In a traditional layered app, one feature is smeared across Controllers, Services, and Repositories folders — to change "place a bid" you touch five places. Slices keep everything for one feature in one folder, so the codebase "screams" what it does and features stay independent (the future-microservice boundaries from brief §8 start here).
- **Where:** It's how your entire backend is organized from now on. Every new feature in every later phase = a new slice.
- **How:** Create `Features/PlaceBid/` and put the command, its validator, its handler, and its minimal-API endpoint inside. Repeat per feature. No shared "Services" dumping ground.

Why / Where / How — MediatR

- **Why:** You need glue that takes an incoming request and finds the one handler that deals with it, so endpoints stay thin and slices stay decoupled.
- **Where:** Between your endpoint and your handler, in every slice.
- **How:** Define a request type (e.g. `PlaceBidCommand`) and a matching `IRequestHandler`. The endpoint calls `mediator.Send(command)`. MediatR routes it.

Why / Where / How — CQRS

- **Why:** Reads and writes have different needs; mixing them creates tangled code. Separating them keeps each simple and sets up the powerful read-model split later (brief §6.2).
- **Where:** Already in your slice names — commands vs queries.
- **How:** Just be disciplined: a command never doubles as a query. The *advanced* read-model version arrives in Phase 6; here you only need the structural split.

Definition of Done:

- ☐ Each feature lives in its own slice folder with everything it needs
- ☐ Endpoints are thin; logic lives in handlers reached via MediatR
- ☐ You can clearly say which slices are commands and which are queries
- ☐ You reproduced the two-bidders race condition, then fixed it with optimistic concurrency, and can explain in your own words what happened

Pitfalls:

- Recreating a giant shared "Services" folder out of habit. Resist it; logic goes in the slice.
- Skipping the concurrency exercise because "it works on my machine." Feeling that bug is the single most valuable 30 minutes in the whole project.

Phase 3 — Serilog (structured logging + your audit trail)

Goal: Replace plain-text logging with structured logging, and start recording every bid as a structured event — the beginning of your tamper-evident audit trail.

Build:

- Wire Serilog in as the app's logger.
- Log each bid as a structured event with fields: `AuctionId`, `BidderId`, `Amount`, `Timestamp`. Not a sentence — *fields*.

Why / Where / How — Serilog

- **Why:** Default logs are flat strings you can only grep. Structured logs are searchable data ("show me every bid over $500 on auction X"). For an auction, that log *is* your audit history. It's also the cheapest high-value win on the whole list — minutes to add.
- **Where:** App-wide, configured at startup. Most valuable inside the `PlaceBid` handler.
- **How:** Add Serilog, set it as the host logger, and log with named properties, e.g. `log.Information("Bid placed {AuctionId} {BidderId} {Amount}", ...)`. Write to the console for now (Phase 10 ships these somewhere searchable).

Definition of Done:

- ☐ Serilog is the app's logger
- ☐ Every bid produces a structured log event with the four fields
- ☐ You can read those events in the console and they're clearly structured, not a flat sentence

Pitfalls:

- Logging secrets or full request bodies. Log the fields you need, nothing sensitive.
- Treating logs as an afterthought — you're intentionally building the audit trail early so Phase 10 has data to show.

Phase 4 — Docker + Docker Compose

Goal: Run your API and Postgres together with a single command, so adding more dependencies later is trivial.

Build:

- A `Dockerfile` for the API.
- A `docker-compose.yml` defining two services: the API and Postgres, wired together.
- `docker compose up` brings the whole thing to life.

Why / Where / How — Docker + Compose

- **Why:** "Works on my machine" disappears when the app and its database run as containers — identical everywhere. And practically: once you can add a service to a compose file, adding Redis (Phase 5) and RabbitMQ (Phase 6) becomes "another few lines," not a fresh ordeal.
- **Where:** It wraps everything you've built. From here, "running the app" means "running the containers."
- **How:** Write a Dockerfile that builds and runs the API. Write `docker-compose.yml` with an `api` service and a `postgres` service; pass the DB connection details via environment variables; put them on the same network so the API can reach Postgres by service name.

Definition of Done:

- ☐ `docker compose up` starts the API and Postgres together
- ☐ The API connects to the containerized Postgres (not a local install)
- ☐ You can stop and restart cleanly, and your migrations apply

Pitfalls:

- Hard-coding `localhost` for the database. Inside Compose, services reach each other by service name, not localhost.
- Fighting Compose for days. If stuck, it's almost always networking or env vars — check those first.

Phase 4.5 — .NET Aspire (orchestration + built-in observability)

Goal: Replace hand-wired Compose plumbing with Aspire's orchestration and get a live dashboard for free — *after* you've felt the manual version, so the magic makes sense.

Build:

- An Aspire **AppHost** project that declares your app and its dependencies (Postgres now; Redis and RabbitMQ as you add them) as resources.
- Aspire starts everything together and injects connection details automatically (service discovery).
- Open the Aspire **dashboard** and watch live logs, traces, and metrics.

Why / Where / How — .NET Aspire

- **Why:** In Phase 4 you manually wrote connection strings and wired services together. Aspire does that for you *and* hands you a real-time dashboard with OpenTelemetry built in. Crucially, doing Compose first means you *understand* what Aspire is automating — you earned the magic.
- **Where:** It becomes the new "front door" for running the system locally. It also quietly sets up the telemetry pipeline that Phase 10 plugs OpenObserve into.
- **How:** Add the AppHost project. Declare resources (the shape is roughly `builder.AddPostgres(...)`, later `builder.AddRedis(...)`, `builder.AddRabbitMQ(...)`) and reference them from your API so connection details flow in automatically. Run the AppHost; explore the dashboard. **Aspire changes fast — confirm current package names and AppHost syntax against the official docs when you reach this phase.**

Definition of Done:

- [ ] Running the AppHost starts the whole system
- [ ] Your API gets its Postgres connection from Aspire, not a hand-written string
- [ ] The Aspire dashboard shows your app's logs and traces live

Pitfalls:

- Skipping Phase 4 to jump straight here. Then Aspire is unexplained magic. Do Compose first.
- Trusting old tutorials for Aspire syntax — it has moved a lot. Use current docs.

Phase 5 — Redis (caching the price + live watchers)

Goal: Make reads instant by caching, and add the live "X people watching" counter.

Build:

- Cache each auction's current price in Redis so the list/detail reads hit memory, not a database join.
- Track **watcher presence**: when someone opens an auction page, record "user is watching auction Y" in Redis with a short expiry; count those to show the watcher number.

Why / Where / How — Redis

- **Why:** Auction prices are read constantly (every card, every refresh, soon every live tick). Reading them from an in-memory store is dramatically faster than recomputing from the database. Redis also naturally handles transient presence data that you don't want cluttering your real database.
- **Where:** It sits beside Postgres as a fast read layer. Postgres stays the source of truth; Redis holds fast copies and ephemeral data.

- **How:** Add Redis as a resource in Aspire. On read paths, check Redis first; on a price change, update Redis. For presence, write a key like `watching:{auctionId}:{userId}` with a short time-to-live that you refresh while the user is on the page; the count of those keys is your watcher count.

Definition of Done:

- [ ] Current price reads come from Redis, with Postgres as the source of truth
- [ ] Opening an auction page increments its watcher count; leaving lets it expire
- [ ] You can feel/measure that cached reads are faster

Pitfalls:

- Letting the cache drift from the truth. Have one clear rule for when Redis is updated (you'll formalize this with events in Phase 6).
- Storing permanent data only in Redis. The database remains the source of truth.

Phase 6 — RabbitMQ + MassTransit (events, the background worker, and the CQRS read model)

Goal: Go event-driven. Bids publish events; other parts react asynchronously. A background worker ends auctions on time. And you build the advanced CQRS read model. This is the meatiest, most valuable phase — **budget two weeks.**

Build:

- Stand up **RabbitMQ** and add **MassTransit** to the app.
- When a bid succeeds, publish a `BidPlaced` event instead of doing everything inline.
- A **consumer** reacts to `BidPlaced` by notifying the now-outbid previous high bidder (the notification itself can just be logged/stored for now).
- A **background worker** watches auction end-times; when one passes, it publishes `AuctionEnded`, which triggers winner notification and a **stubbed** payment step.
- **The CQRS read model (brief §6.2):** a handler reacts to `BidPlaced` by updating the `AuctionCard` projection in Redis (new price, bid count + 1). Reads now serve from this projection.

Why / Where / How — RabbitMQ

- **Why:** Some work shouldn't block the bid. Sending an "outbid" notification inline makes the bidder wait for unrelated work and couples your features together. A message broker lets the bid finish instantly while notifications happen separately — and it's the *seam* that lets you split into microservices later (brief §8).
- **Where:** In the middle, between the part that places bids and the parts that react (notifications, projections, payments).
- **How:** Run RabbitMQ (add it as an Aspire resource). Your `PlaceBid` handler *publishes* an event; separate consumers *subscribe* to it.

Why / Where / How — MassTransit

- **Why:** Talking to RabbitMQ directly means handling connections, channels, serialization, and retries by hand. MassTransit gives you strongly-typed C# messages and consumers, plus retry/error handling out of the box — and it sets up the queue topology for you.
- **Where:** It's the library wrapping all your publish/consume code.
- **How:** Register MassTransit with the RabbitMQ transport at startup and let it auto-configure endpoints. Define events as plain C# records (`BidPlaced`, `AuctionEnded`). Publish with the publish endpoint; consume by implementing a consumer per event. Use its test harness to test consumers without a live broker.

Why / Where / How — CQRS read model (the advanced half)

- **Why:** Computing the auction card from a live join on every read won't scale. Maintaining a ready-made projection on write makes reads instant and lets read/write scale independently (your stated goal).
- **Where:** The write side stays Postgres; the read side becomes the `AuctionCard` in Redis, kept fresh by event handlers.
- **How:** A consumer of `BidPlaced` updates the Redis projection. Your read queries now return the projection, not a fresh join.

Definition of Done:

- ☐ A successful bid publishes `BidPlaced`; the bid response no longer waits on notification work
- ☐ A consumer reacts to `BidPlaced` and produces an "outbid" notification (logged/stored)
- ☐ A background worker ends auctions at `EndsAt`, publishing `AuctionEnded` → winner notified + payment stub runs
- ☐ The `AuctionCard` read model in Redis is updated by events, and reads serve from it
- ☐ You can explain, in your own words, the path from "place bid" to "everyone's card is fresh"

Pitfalls:

- Rushing this phase. It's the highest-value concept on your whole list. Two weeks is fine.
- Making consumers depend on each other. Each consumer reacts to an event independently — that independence is the point.
- Forgetting bids for the same auction can still race. Process them in a way that serializes per auction (per-auction ordering), building on your Phase 2 concurrency fix.

Phase 7 — SignalR (the live payoff)

Goal: The moment the project becomes *fun* — bids appear on everyone's screen instantly, no refresh.

Build:

- A SignalR **hub** clients connect to, grouped per auction (everyone viewing auction Y joins group Y).
- When `BidPlaced` is handled (Phase 6), broadcast the new price + bid count to that auction's group.

- The Angular detail page subscribes and updates the price live; the watcher count (Phase 5) also updates live.

Why / Where / How — SignalR

- **Why:** A bid that doesn't update other people's screens is a broken auction. SignalR pushes updates from server to browser in real time, so you don't poll. This is the single feature that makes the whole app feel alive.
- **Where:** Between your backend (specifically the `BidPlaced` handling from Phase 6) and every connected Angular client viewing that auction.
- **How:** Add a SignalR hub on the API. Clients viewing an auction join that auction's group. When a bid is processed, broadcast to the group. In Angular, connect to the hub, join the auction's group on page open, and update the UI on each pushed message.

Definition of Done:

- ☐ Open the same auction in two browser windows; bid in one; the price jumps in the other with no refresh
- ☐ The live broadcast is driven by the Phase 6 event flow, not a separate ad-hoc path
- ☐ Watcher count updates live too

Pitfalls:

- Broadcasting to *all* clients instead of just the relevant auction's group — use groups.
- Bypassing your event flow and broadcasting straight from the endpoint. Keep one source of truth: the `BidPlaced` handling.

Phase 8 — Keycloak (identity & roles)

Goal: Real login, logout, and roles — sellers, bidders, admins — without building authentication yourself.

Build:

- Run **Keycloak** and configure a realm with roles (Bidder, Seller, Admin) and a client for your app.
- Angular redirects users to Keycloak to log in; the API validates the resulting token and enforces roles.
- Bids and auctions now tie to the authenticated user. Add "My bids", "My listings", "Watching".

Why / Where / How — Keycloak

- **Why:** Authentication is security-critical and easy to get dangerously wrong. Keycloak is a mature identity server that handles login, logout, tokens, and roles via the standard OpenID Connect protocol, so you don't hand-roll auth. It's placed late because it's fiddly and you now have the Docker/Aspire skills to handle it.
- **Where:** It owns identity. Your Angular app sends users there to log in; your API trusts the tokens it issues and checks roles on protected endpoints.

- **How:** Run Keycloak (an Aspire/Compose resource). Create a realm, roles, and a client. Configure Angular to use OpenID Connect against Keycloak. Configure the API to validate Keycloak's tokens and restrict actions by role (only Sellers create auctions, only Admins see the audit log, etc.).

Definition of Done:

- [ ] Users log in and out via Keycloak
- [ ] The API rejects unauthenticated calls to protected endpoints
- [ ] Roles are enforced (a Bidder can't create an auction; only Admin sees admin views)
- [ ] Bids and listings are tied to the logged-in user; "My bids"/"My listings" work

Pitfalls:

- Expecting it to "just work" quickly — OpenID Connect has a learning curve. Follow a current step-by-step ASP.NET Core + Keycloak guide.
- Trying to also keep your old homemade user table as the source of identity. Keycloak owns identity now; your `User` becomes a thin profile linked by Keycloak's user ID.

Phase 9 — HashiCorp Vault (secrets)

Goal: Get every secret out of config files and into a proper secrets manager.

Build:

- Run **Vault**.
- Move the database connection string, the Keycloak client secret, and any email/API keys out of `appsettings`/env and into Vault.
- The app reads its secrets from Vault at startup.

Why / Where / How — HashiCorp Vault

- **Why:** Real money and real identity mean real secrets, and secrets in config files (or worse, Git history) are a serious risk. Vault stores them securely and hands them out only to authorized apps. It's a production-maturity concern, so it comes once the app actually has secrets worth protecting.
- **Where:** Between your app's startup configuration and every secret it needs.
- **How:** Run Vault (Aspire/Compose resource), store your secrets in it, and configure the app to fetch them from Vault on startup instead of reading them from plain config. Use the hands-on HashiCorp "Learn" tutorials for the exact flow.

Definition of Done:

- [ ] No real secret remains in `appsettings` or committed config
- [ ] The app fetches its secrets from Vault at startup and runs normally
- [ ] You can explain why config-file secrets are risky

Pitfalls:

- Committing a secret "just temporarily." Once it's in Git history, it's compromised. Use Vault from the start of this phase.

- Over-scoping — move the handful of secrets you actually have; you're learning the pattern, not securing a bank.

Phase 10 — OpenObserve (observability & the trust loop)

Goal: Ship your logs/traces/metrics somewhere you can search and watch — and use it to surface suspicious bidding. This closes the loop on the audit trail you started in Phase 3.

Build:

- Run **OpenObserve**.
- Point your telemetry (the Serilog bid-audit logs from Phase 3, plus the OpenTelemetry traces/metrics Aspire set up in Phase 4.5) at OpenObserve.
- Build a view/query to explore bid activity and spot patterns — e.g. one account hammering one auction.

Why / Where / How — OpenObserve

- **Why:** Logs scrolling past in a console are useless at scale. An observability platform lets you store, search, and visualize what your running system is doing — and for an auction, that means an auditable, searchable bid history and the ability to spot abuse. It's the capstone because it consumes everything the earlier phases produced.
- **Where:** At the end of your telemetry pipeline — the destination your Serilog logs and Aspire/OpenTelemetry traces flow into.
- **How:** Run OpenObserve (Aspire/Compose resource). Configure your log sink and the OpenTelemetry exporter to send data to it (OTLP). Because Aspire already wired telemetry in Phase 4.5, this is mostly "point the exporter at OpenObserve," not "set up telemetry from scratch." Then build a saved query/dashboard over your bid events.

Definition of Done:

- ☐ Your structured bid logs and traces land in OpenObserve and are searchable
- ☐ You have a view that shows bid activity per auction
- ☐ You can construct a query that would surface a suspicious pattern (one account, many rapid bids on one auction)

Pitfalls:

- Treating it as "just logs." The value is *searching and correlating* — practice writing a query that answers a real question.
- Re-doing telemetry from scratch. Lean on what Aspire already set up in Phase 4.5.

Appendix A — The scale-up endgame (optional final boss)

Once Phase 10 is done, you have a polished modular monolith. To take it to true microservices:

1. Pick one slice group from brief §8 — **Notifications** is the easiest first cut, because it only *consumes* events and shares no database tables.
2. Move that slice group into its own deployable service project.
3. It already talks to the rest of the system only through the message broker, so it keeps working — that's the payoff of the event-driven seam you built in Phase 6.
4. Add it to your Aspire AppHost as a separate service. Aspire makes running 4–5 services locally almost as easy as running 1.
5. Repeat for **Payments**, then **Bidding** and **Auctions**.

Do this **only after** the monolith is complete. The monolith is not a throwaway — it's the correct first step, and the split is mechanical because of how you designed it.

Appendix B — Master checklist (tick as you go)

- ☐ Phase 0 — Setup & foundations
- ☐ Phase 1 — Core canvas (API + Postgres + Angular, refresh-to-see bids)
- ☐ Phase 2 — Vertical slices + MediatR + CQRS + concurrency fix
- ☐ Phase 3 — Serilog structured logging / audit trail
- ☐ Phase 4 — Docker + Compose
- ☐ Phase 4.5 — .NET Aspire orchestration + dashboard
- ☐ Phase 5 — Redis caching + watcher presence
- ☐ Phase 6 — RabbitMQ + MassTransit + background worker + CQRS read model
- ☐ Phase 7 — SignalR live bids
- ☐ Phase 8 — Keycloak auth & roles
- ☐ Phase 9 — Vault secrets
- ☐ Phase 10 — OpenObserve observability
- ☐ Appendix A — split into microservices (optional)

Resources (pull current versions when you reach each phase)

Vertical Slice Architecture

• Milan Jovanović's vertical slice articles (the go-to explanation, with MediatR + REPR pattern)

- Nadir Badnjevic's `VerticalSliceArchitecture` GitHub template — a runnable .NET template; clone it and study one slice

MassTransit + RabbitMQ

- Official MassTransit RabbitMQ quick start (zero-to-running, includes a preconfigured Docker image)
- Rahul Nath's beginner MassTransit + RabbitMQ guide (explains the queue topology most tutorials skip)

Everything else — go to the official docs

- Serilog → serilog.net
- Redis with .NET → Microsoft Learn "distributed caching in ASP.NET Core"
- Docker → Microsoft Learn "Containerize a .NET app"
- .NET Aspire → official Aspire docs (verify current syntax — it changes fast)
- SignalR → Microsoft Learn's official SignalR tutorial (one of their best)
- Keycloak → official Keycloak "Getting Started", plus a current ASP.NET Core OIDC integration walkthrough
- HashiCorp Vault → HashiCorp's hands-on "Learn" tutorials
- OpenObserve → official OpenObserve docs
- Angular → angular.dev

Educators worth following across the whole stack: Milan Jovanović, Nick Chapsas, Les Jackson, Rahul Nath.

> Filter tip: prefer resources dated 2024–2026. .NET, Aspire, minimal APIs, and MassTransit have all shifted recently, and older tutorials will have you writing outdated startup code.

End of Build Roadmap. Build in order, tick every Definition of Done, and you won't get lost in the middle.